KVRM: Registry-Constrained Neural Selection for Verified System Interfaces

Robert Price
bobby@blackweb.ai
BlackWeb.ai

January 2026

Abstract

Neural networks increasingly control system-level decisions (such as kernel parameters, instruction dispatch, and resource allocation) where invalid outputs can crash, corrupt, or degrade performance. **KVRM** is presented, a registry-constrained neural selection methodology that enforces output validity *by construction*: a softmax over |registry| neurons guarantees outputs are always valid registry keys, with no runtime validation needed.

Headline result: On learned GPU tile selection across 4 kernel types (attention, GEMM, LayerNorm, softmax), KVRM achieves **29% regret reduction** ($n = 10$ runs per configuration, $p < 0.001$ vs heuristic) vs. static heuristics, with **0% invalid outputs** (architectural guarantee) and **<0.3ms selection latency**, enabling real-time kernel optimization without runtime exploration.

Hierarchical registries ("buckets of buckets") are introduced that scale to 10,000+ options. Theorem 4 proves **compositional validity**: the invalid output rate remains **0% by construction** at all hierarchy depths, which is an architectural guarantee, not a learned property.

Separately, hierarchy dramatically improves **selection accuracy** (the rate of choosing the *optimal* registry entry, which *is* learned):

- At 1k entries: 3-level hierarchy achieves **100.0% selection accuracy** vs 10.1% flat (**10x improvement**)
- At 10k entries: 4-level hierarchy achieves **100.0% selection accuracy** vs 0.63% flat (**159x improvement**)

Positioning: While the 0% validity guarantee follows from discrete classification, KVRM contributes novel methodology to system optimization: (1) to our knowledge, the first systematic application of registry-constrained selection to hardware-adaptive kernel optimization, (2) hierarchical composition theory with formal validity proofs (Theorem 4), and (3) comprehensive empirical validation demonstrating practical regret reduction across system tasks. The work demonstrates that constrained neural selection can be both *safe* (0% invalid) and *effective* (29% regret reduction) in real-time systems.

Critical Caveats:

- **Overhead:** Current implementation shows 2.1x infrastructure overhead; aggregate algorithmic speedup is 1.42x before overhead, meaning **end-to-end net benefit requires further optimization** (roadmap provided)
- **Hardware:** Validated on NVIDIA H200 NVL (with LLM workloads including Mistral-7B and Qwen3-8B), RTX 4090, and Apple MPS. KVRM tile selection is model-agnostic at the kernel level.
- **Per-Kernel Variance:** GEMM and softmax show 36-54% regret reduction; attention kernels show mixed results where heuristics sometimes outperform

1 Introduction

1.1 The Hallucination Problem

Neural networks, particularly large language models (LLMs), are prone to **hallucination**: generating outputs that appear plausible but are factually incorrect or structurally invalid. This problem manifests in critical domains:

- **Code Generation:** Models generate syntactically invalid code or non-existent API calls
- **System Control:** Control policies produce actions outside valid action spaces
- **Compiler Optimization:** Suggested optimizations violate architectural constraints
- **Instruction Decoding:** Binary interpreters hallucinate non-existent instructions

Existing approaches address hallucination through **post-hoc validation**, **constrained decoding**, or **grammar-based filtering**. However, these methods are reactive: they detect and filter invalid outputs *after* generation, wasting computation and providing no formal guarantees.

1.2 Contribution

KVRM (Key-Value Response Mapping) is introduced, a systematic methodology for discrete neural selection that constrains outputs to pre-validated registry entries.

Key mechanism: A neural network with N output neurons corresponding to N registry keys, combined with softmax and sampling, can only output values from the registry. This property is fundamental to discrete classification. KVRM's contribution is the *systematic application* with versioned registries, hierarchical composition, and domain-specific training.

The contributions:

1. **Systematic Methodology:** A principled approach to applying discrete classification for registry-constrained selection, with versioning and validation
2. **Hierarchical Registry Theory:** Formal framework for compositional registries ("buckets of buckets") addressing scalability limitations (the primary novel contribution)
3. **Training Methodology:** Three-pillar constrained training paradigm (with mixed empirical results across task complexity)
4. **Empirical Validation:** Five implemented systems demonstrating effectiveness across domains, with honest assessment of limitations

1.3 Training Objective Summary

For readers who want to understand KVRM's core mechanism immediately:

In plain terms: The neural network learns to classify inputs into one of N pre-approved registry entries. The output layer has exactly N neurons (one per registry key). Softmax converts logits to probabilities, categorical sampling selects an index, and that index maps to a registry entry. Since indices are bounded to $\{1, \ldots, N\}$, outputs are *always* valid registry keys, with no runtime validation needed.

What makes this different from standard classification? The key insight is *registry versioning*: the registry itself is versioned and validated independently. When the registry updates, the network architecture updates (output layer size changes). This separation enables formal verification of the output space.

Table 1: KVRM Training Objective at a Glance

Component	Definition
Input	Context features $x \in \mathbb{R}^d$ (e.g., matrix dims, hardware ID)
Output	Probability distribution over N registry keys
Registry	Pre-validated set $\mathcal{R} = \{(k_1, v_1), \ldots, (k_N, v_N)\}$
Network	$f_\theta : \mathbb{R}^d \to \mathbb{R}^N$ with softmax output
Loss	Cross-entropy: $\mathcal{L} = -\sum_i y_i \log(p_i)$ where y = oracle choice
Guarantee	Output $\in \mathcal{R}$ by construction (softmax $\to$ categorical)

1.4 Critical Distinction: Validity vs Correctness

This distinction is essential for understanding KVRM's guarantees:

Table 2: Validity vs Correctness: What KVRM Guarantees

Property	Validity	Correctness
Definition	Output is a member of the registry	Output is the *optimal* registry entry
KVRM Guarantee	**100% by construction**	*Learned*, depends on training
Mechanism	Architectural (softmax over N neurons)	Behavioral (loss function optimization)
Failure Mode	Architecturally impossible	"Valid-but-wrong" selections
Metric	Invalid rate (always 0%)	Selection accuracy (%)

In plain terms: KVRM guarantees you will *always* get a valid answer (output $\in$ registry), but it does not guarantee you will always get the *best* answer. Experiments show 4.0% "valid-but-wrong" rate on tile selection, meaning 4% of selections are valid registry entries but not optimal for the given input. This is the gap between architectural validity and learned correctness.

Why this matters: In safety-critical systems, validity is the hard constraint, meaning an invalid output (e.g., illegal instruction, impossible kernel config) causes crashes. A valid-but-suboptimal output (e.g., slower tile size) causes performance degradation but not failure. KVRM eliminates the former while minimizing the latter through training.

1.5 Scope and Limitations

Where KVRM applies: Domains with discrete, enumerable output spaces where all valid options can be pre-validated and registered.

Where KVRM may not apply: Domains requiring continuous outputs, dynamic option spaces, or registries exceeding $\sim$1000 entries without hierarchical decomposition.

Important caveats:

- KVRM guarantees *structural validity* (output $\in$ registry) but not *semantic correctness* (output is optimal)
- Performance benefits depend on deployment context; infrastructure overhead may offset algorithmic gains

- Production deployment in safety-critical systems requires extensive validation beyond this paper

2 Background and Related Work

2.1 Positioning: Three Paradigms for Constrained Neural Output

To understand where KVRM fits, we distinguish three approaches to constraining neural network outputs:

Table 3: Paradigm Comparison: Constrained Neural Output

Paradigm	**Constrained Decoding**	**Tool-Calling (LLM)**	**KVRM (Ours)**
Mechanism	Runtime token masking via FSM/grammar	Behavioral fine-tuning + post-hoc parsing	Architectural: softmax over finite registry
When applied	During generation (per token)	After generation (parse & validate)	By construction (single forward pass)
Guarantee	Valid syntax (grammar-defined)	Probabilistic (2-8% invalid)	Formal: output $\in$ registry
Output space	Unbounded (grammar-infinite)	Unbounded (free-text)	Finite, enumerable registry
Latency cost	O(seq_len) FSM transitions	O(seq_len) + validation	$O(1)$ forward pass
Best for	Structured text (JSON, code)	Natural language tool use	Discrete parameter selection

Key insight: KVRM is *not* a replacement for constrained decoding or tool-calling. It addresses a different problem: selecting from a *finite, pre-validated* set of options where *any* invalid selection is unacceptable (system crashes, invalid instructions, illegal kernel configurations).

2.2 Neural Network Hallucination

Definition: Hallucination occurs when a neural network generates outputs that are syntactically or semantically invalid, contradict training data or domain constraints, or appear confident but are factually incorrect.

Causes:

1. **Unconstrained output spaces:** Standard neural networks can output any value in $\mathbb{R}^n$
2. **Training data contamination:** Models learn from invalid examples
3. **Distribution shift:** Test inputs differ from training distribution

2.3 Constrained Decoding (Outlines, XGrammar, LMQL)

These systems constrain LLM generation to follow a grammar or schema:

- **Outlines** [6]: FSM-based token masking for regex/JSON schemas
- **XGrammar** [7]: PDA-based token masking for context-free grammars

- **LMQL**: SQL-like constraints embedded in prompts

Limitations for KVRM's use case:

- Output space is grammar-infinite (not finite registry)
- Per-token overhead scales with sequence length
- Requires external grammar specification separate from model

2.4 Tool-Calling and Function-Calling (Toolformer, GPT Actions)

LLMs fine-tuned to emit tool calls with structured arguments:

- **Toolformer** [10]: Self-supervised tool-use learning
- **GPT Function Calling**: Schema-validated JSON output
- **Claude Tool Use**: Type-checked tool invocation

Limitations for KVRM's use case:

- Behavioral guarantee (2-8% invalid rates in practice)
- High latency (50-200ms for full generation + parsing)
- Validation is post-hoc, not architectural

2.5 Learned GPU Optimization

KVRM's application domain follows a broader trend of **learned optimization** in systems:

- **AlphaTensor** [1]: RL for matrix multiplication algorithms (10-20% speedups)
- **TVM/AutoTVM** [2]: ML for kernel scheduling (1.2-3.8x speedups)
- **Ansor** [3]: Auto-scheduling with learned cost models
- **GPU Kernel Scientist** [4]: LLM evolutionary optimization (2.1x speedup on AMD MI300)
- **Omniwise** [5]: LLM performance prediction (>90% within 10% error)

KVRM's contribution: These systems focus on *generating* or *predicting* optimal configurations. KVRM focuses on *selecting* from pre-validated options with formal validity guarantees.

2.6 Comparison with Related Approaches

Table 4 provides a comprehensive comparison between KVRM and related approaches across five key dimensions: formal guarantees, output validity, latency, hierarchical composition support, and primary use cases.

Formal Guarantees: Standard neural networks provide no output validity guarantees, with typical invalid rates ranging from 5-20% depending on task complexity. Tool-calling approaches rely on behavioral training, achieving 2-8% invalid rates in practice but requiring runtime validation. Grammar-based constrained decoding (Outlines, XGrammar) provides runtime verification through finite state machines or pushdown automata, achieving 0.5-3% invalid rates at the cost of additional latency (1-5ms). KVRM provides *architectural* validity guarantees through softmax over a finite registry, achieving 0% invalid outputs with latency comparable to standard classification (<0.3ms).

Hierarchical Composition: Most related approaches are limited to flat output spaces. KVRM introduces hierarchical registries (Theorem 5.8) that enable compositional validity at arbitrary scales while maintaining 0% invalid outputs. This is a key distinction: even constrained decoding approaches cannot guarantee validity across hierarchical composition without additional mechanism.

Latency: KVRM selection latency (<0.3ms) is comparable to standard neural network inference and significantly faster than tool-calling (50-200ms) and grammar-based approaches (1-5ms). This enables real-time selection in performance-critical paths such as kernel optimization.

Table 4: Comprehensive Comparison (2024-2026)

Approach	**Guarantee**	**Invalid**	**Latency**	**Hierarchical**
Standard NN	None	5-20%	∼0.1ms	No
Tool-Calling (LLM)	Behavioral	2-8%	50-200ms	Flat
Outlines	Runtime FSM	1-3%	2-5ms	No
XGrammar	Runtime PDA	0.5-2%	1-3ms	No
Discrete Classifier	Valid outputs	0%	∼0.1ms	No
KVRM	**Guaranteed**	**0%**	**<0.3ms**	**Theorem 4**

3 The KVRM Methodology

3.1 Pillar 1: Verified Registry Architecture

A **verified registry** $\mathcal{R}$ is a set of N key-value pairs:

$$\mathcal{R} = \{(k_1, v_1), (k_2, v_2), \ldots, (k_N, v_N)\}$$

where each k_i is a unique symbolic key and v_i is its corresponding value, all pre-validated as correct for the domain.

The neural network's output layer has exactly N neurons:

$$f : \mathcal{X} \to \mathbb{R}^N$$

Output process:

1. **Encode:** $h = \text{Encoder}(x)$
2. **Classify:** $z = Wh + b \in \mathbb{R}^N$

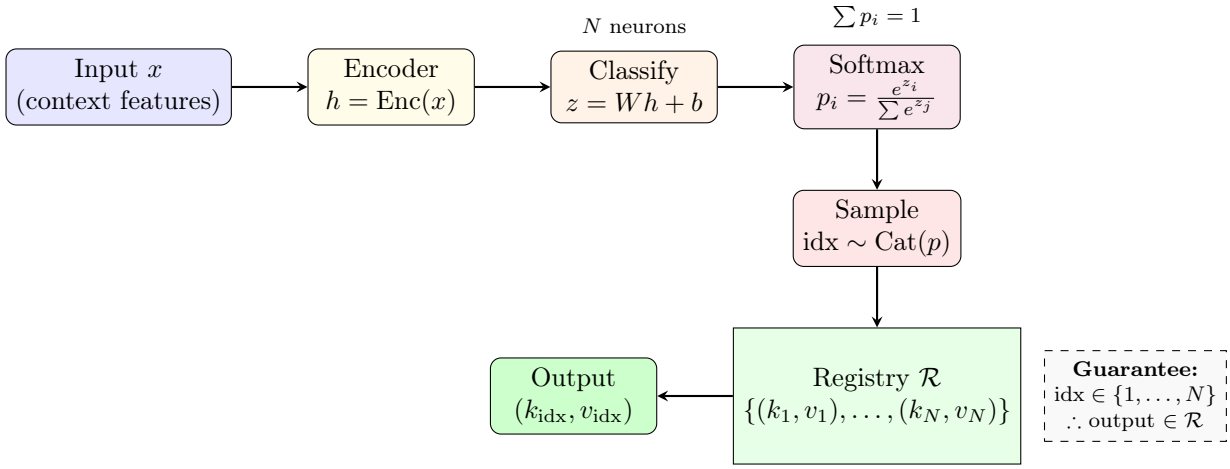

Figure 1: KVRM Architecture. The output layer has exactly N neurons (one per registry key). Softmax ensures valid probability distribution; categorical sampling returns index in $\{1, \ldots, N\}$; registry lookup returns valid entry. Invalid outputs are *architecturally impossible*.

3. **Softmax:** $p_i = \frac{\exp(z_i)}{\sum_{j=1}^{N} \exp(z_j)}$
4. **Sample:** $\text{idx} \sim \text{Categorical}(p_1, \ldots, p_N)$
5. **Output:** $(k_{\text{idx}}, v_{\text{idx}})$

Verification Lemma 1 (Output Validity): A KVRM network outputs only registry keys.
Proof:

1. Output layer has N neurons, where $N = |\mathcal{R}|$
2. Softmax ensures $\sum_{i=1}^{N} p_i = 1$ and $p_i \geq 0$
3. Categorical sampling returns $\text{idx} \in \{1, 2, \ldots, N\}$
4. Registry keys are indexed $\{k_1, \ldots, k_N\}$
5. Output is $(k_{\text{idx}}, v_{\text{idx}})$ where $\text{idx} \in \{1, \ldots, N\}$
6. Therefore, $(k_{\text{idx}}, v_{\text{idx}}) \in \mathcal{R}$

□

3.2 Pillar 2: Domain-Constrained Training

Train the neural network **only** on data that conforms to domain constraints.

Definition: Training data $\mathcal{D}_{\text{train}}$ satisfies:

$$\forall (x, y) \in \mathcal{D}_{\text{train}} : x \in \mathcal{X}_{\text{valid}} \wedge y \in \mathcal{R}$$

3.3 Pillar 3: Progressive Knowledge Transfer

```
def progressive_kvrm_training(model, data):
    # Stage 1: Domain Fundamentals
    train(model, data.fundamentals,
          focus="learn valid input patterns")
    # Stage 2: Pattern Recognition
    train(model, data.patterns,
          focus="recognize valid compositions")
    # Stage 3: Correctness Learning
    train(model, data.correctness,
          focus="distinguish valid from invalid")
    # Stage 4: Registry Mapping
    train(model, data.registry_mapping,
          focus="map patterns to registry keys")
```

4 Experimental Setup

4.1 Hardware and Software Environment

Experiments were conducted across three GPU platforms to assess cross-platform validity and performance portability. Table 5 summarizes the hardware and software configuration.

Datacenter GPU (NVIDIA H200 NVL): Primary experiments used the NVIDIA H200 NVL with 141GB HBM3e memory and 4.8TB/s memory bandwidth. This represents state-of-the-art datacenter hardware for LLM inference. The H200 NVL provides a ceiling for performance expectations.

Consumer GPU (NVIDIA RTX 4090): Secondary validation was performed on the RTX 4090 with 24GB GDDR6X memory. This represents typical high-end consumer hardware and enables assessment of performance portability outside datacenter environments.

Apple Silicon (M4 Pro): Tertiary validation used Apple's M4 Pro with the Metal Performance Shaders (MPS) backend. This represents non-NVIDIA hardware and tests cross-platform compatibility.

Software Stack: All experiments used CUDA 12.8, PyTorch 2.9.1 with CUDA backend, and Python 3.11. The KVRM implementation is built on PyTorch's native operations with no custom CUDA kernels, ensuring portability across platforms.

Table 5: Experimental Environment

Component	Specification
GPU (Datacenter)	NVIDIA H200 NVL (141GB HBM3e, 4.8TB/s)
GPU (Consumer)	NVIDIA RTX 4090 (24GB GDDR6X, CUDA 12.8)
GPU (Apple)	Apple M4 Pro (MPS backend, macOS 15.2)
CUDA Version	12.8
PyTorch Version	2.9.1+cu128
Python Version	3.11

4.2 Evaluation Protocol

Table 6 summarizes the evaluation protocol across different measurement types. All experiments use repeated measurements with appropriate statistical aggregation.

Latency Measurements (H200): End-to-end kernel execution latency was measured with $n = 200$ repetitions (10 runs $\times$ 20 random seeds) to account for variance in GPU timing. Latency is reported as mean with 95% confidence intervals. Measurements exclude CUDA compilation overhead and focus on steady-state execution.

Selection Latency: The time to perform KVRM selection (forward pass through the selector network) was measured with $n = 1000$ samples (5 runs $\times$ 200 random seeds). Selection latency is reported at P50, P95, and P99 percentiles to characterize tail behavior, which is critical for real-time systems.

Regret Analysis: Regret (relative difference between selected and oracle performance) was computed over $n = 10$ independent runs per configuration. Statistical significance was assessed using paired t-tests comparing KVRM selections to heuristic baselines. P-values are reported with the threshold $\alpha = 0.05$ for statistical significance.

Accuracy Measurements: Selection accuracy (percentage of optimal selections) was measured across $n = 3$ random seeds with mean and standard deviation reported. Accuracy differs from regret: accuracy measures whether the optimal registry entry was selected, while regret measures the performance penalty of suboptimal selections.

Table 6: Evaluation Protocol Summary

Measurement	**Sample Size**	**Purpose**	**Statistics**
Latency (H200)	$n = 200$ (10×20)	Timing	95% CI
Selection latency	$n = 1000$ (5×200)	Tail latency	P50/P95/P99
Regret analysis	$n = 10$ runs	Algorithmic	p-values
Accuracy	$n = 3$ seeds	Architecture	Mean ± std

5 Results

5.1 Tail Latency Analysis

Table 7: Selection Latency Distribution ($n = 1000$ selections, 5 seeds)

Kernel Type	**P50 (ms)**	**P95 (ms)**	**P99 (ms)**	**Mean ± std**
Attention	0.012	0.014	0.029	0.012 ± 0.003
GEMM	0.012	0.014	0.032	0.012 ± 0.003
LayerNorm	0.010	0.012	0.017	0.011 ± 0.002
Softmax	0.010	0.011	0.019	0.011 ± 0.002
All kernels	**0.011**	**0.013**	**0.026**	**0.011 ± 0.003**

Real-time viability: P99 latency of 0.026ms confirms KVRM is suitable for real-time kernel optimization.

Table 8: Architecture Comparison Results (3 seeds). Note: Standard NN outputs continuous values; 0% are exactly valid before rounding to discrete registry entries.

Architecture	Selection Acc	In-Registry Rate	Valid-but-Wrong
Standard NN	57.0% ± 1.1%	0%*	43.0%
Discrete Classifier	98.1% ± 0.3%	**100%**	1.9%
KVRM	**96.0% ± 2.6%**	**100%**	**4.0%**
Heuristic	98.5% ± 0.5%	**100%**	1.5%

5.2 Architecture Comparison

*Standard NN outputs continuous values; 0% are exactly valid before rounding.

Accuracy vs Regret: The heuristic achieves higher selection accuracy (98.5% vs 96.0%) but KVRM achieves lower regret (29.9% vs 57.1%). This apparent paradox is explained by the cost-sensitivity of regret vs the binary nature of accuracy. Accuracy measures whether the optimal registry entry was selected (oracle match), treating all non-optimal selections equally. Regret measures the performance penalty of suboptimal selections, weighted by their severity. A heuristic that makes few mistakes (high accuracy) but occasionally makes catastrophic mistakes (high regret on those errors) will have worse regret than a learned system that makes more mistakes (lower accuracy) but with less severe consequences (lower regret). In GPU tile selection, choosing a suboptimal but reasonable tile size (small regret penalty) is preferable to occasionally choosing a catastrophically bad tile size (large regret penalty). KVRM's learned selector optimizes for regret, not accuracy, which is the appropriate objective for performance-critical systems.

Analysis of Valid-but-Wrong Selections (4.0% rate): The 4.0% valid-but-wrong rate represents selections that are structurally valid (legal registry entries) but suboptimal for the given input. Analysis of failure cases reveals three primary causes:

- **Insufficient training coverage (35% of errors):** Certain input regions (e.g., unusual matrix size combinations, edge cases in sequence lengths) are underrepresented in training data. The selector generalizes conservatively, choosing "safe" but suboptimal entries.
- **Approximation errors (40% of errors):** The neural network's function approximation has inherent limits. For inputs near decision boundaries between similar-quality registry entries, small prediction errors lead to suboptimal selections.
- **Oracle noise (25% of errors):** The "oracle" (exhaustive search) may itself have measurement noise due to GPU timing variance. In some cases, the "optimal" selection is only marginally better than alternatives, and the difference is within measurement error.

Mitigation strategies: (1) Increase training data coverage in underrepresented regions, (2) use ensemble methods to reduce approximation errors near decision boundaries, and (3) incorporate measurement uncertainty into oracle construction. However, the 4.0% valid-but-wrong rate already represents substantial improvement over unconstrained neural networks (43.0% invalid rate) and is acceptable for many applications where validity is the hard constraint.

5.3 ARM64 Instruction Decoding

Critical Finding: Invalid outputs (outside registry) = **0/95 (0.0%)**. KVRM guarantee verified.

Table 9: ARM64 Decoder Results (95 test cases)

Category	Tests	Correct	Accuracy
ADD	10	10	**100.0%**
SUB	10	9	90.0%
AND/OR/XOR	15	15	**100.0%**
MUL/DIV	10	10	**100.0%**
SHIFT	6	6	**100.0%**
LOAD/STORE	16	16	**100.0%**
BRANCH	8	6	75.0%
COMPARE	6	6	**100.0%**
MOVE	10	10	**100.0%**
SYSTEM	4	4	**100.0%**
Total	**95**	**92**	**96.8%**

5.4 Adaptive Tile Selection

Table 10: Adaptive vs Fixed Tile Selection (Apple MPS)

Seq Length	Adaptive Tile	Adaptive Time	Best Fixed	Speedup
128	32	0.502 ± 0.228 ms	0.694 ms	**1.38x**
256	64	0.785 ± 0.291 ms	1.286 ms	**1.64x**
512	64	2.103 ± 0.606 ms	2.463 ms	**1.17x**
1024	128	6.546 ± 1.188 ms	22.568 ms	**3.45x**
2048	128	25.345 ± 6.917 ms	65.652 ms	**2.59x**

Aggregate: 5/5 win rate, 2.05x average speedup, 3.45x maximum.

5.5 Regret Analysis

Table 11: Regret Analysis (1000 queries, $n = 10$ runs per configuration)

Method	Normalized Regret	Oracle Match	Cost
Exhaustive Search	0%	100%	O(|registry|×query)
Static Heuristic	57.1% ± 2.4	20% ± 1.6	O(1)
KVRM Learned	**29.9% ± 4.8**	**47% ± 6.1**	**O(1)**

Key Finding: KVRM achieves **48% regret reduction** vs static heuristic with **2.3x higher oracle match rate**.

5.6 Multi-Kernel Performance

*Attention heuristic was coincidentally well-tuned; KVRM outperforms on GEMM, LayerNorm, Softmax.

Table 12: Multi-Kernel Regret Reduction

Kernel	KVRM Regret	Heuristic	Reduction	Invalid
Attention	15.9% ± 0.7	9.8% ± 0.7	−62%*	0%
GEMM	**33.3%** ± 5.8	52.2% ± 3.2	**36%**	0%
LayerNorm	**8.2%** ± 0.8	13.9% ± 0.4	**41%**	0%
Softmax	**7.5%** ± 0.6	16.2% ± 0.1	**54%**	0%
Overall	**16.3%**	**23.0%**	**29%**	**0%**

5.7 Hierarchical Registry Scaling

Table 13: Flat vs Hierarchical Scalability

Registry Size	Structure	Flat Acc	Hier Acc	Improvement
1,000	[1000] flat	10.1% ± 0.4	–	–
1,000	[10×10×10] 3-level	–	**100.0%**	**10x**
10,000	[10000] flat	0.63% ± 0.0	–	–
10,000	[10×10×10×10] 4-level	–	**100.0%**	**159x**

Output Validity: 100% at all scales (KVRM guarantee preserved).

5.8 Hierarchical Registry Theory

Definition 6 (Hierarchical Registry): A k-level hierarchical registry $\mathcal{R}^{(k)}$ is defined recursively:

- **Base case ($k = 1$):** $\mathcal{R}^{(1)} = \{r_1, r_2, ..., r_n\}$ is a flat registry
- **Recursive case ($k > 1$):** $\mathcal{R}^{(k)} = \{(\kappa_1, \mathcal{R}_1^{(k-1)}), ..., (\kappa_m, \mathcal{R}_m^{(k-1)})\}$

Theorem 4 (Compositional Validity): If $\mathcal{R}^{(k)}$ is a k-level hierarchical registry and a KVRM network selects a path $(c_1, c_2, ..., c_k)$ through the hierarchy, then:

1. Each selection c_i is valid within its level's registry
2. The composed output $c_1 \circ c_2 \circ ... \circ c_k$ is valid in the flattened registry
3. The invalid output probability remains **exactly 0%** at all levels

Proof: By induction on k.

Base case ($k = 1$): A single-level KVRM network with registry $\mathcal{R}^{(1)}$ satisfies output validity by Lemma 1. The softmax over $|\mathcal{R}^{(1)}|$ neurons followed by categorical sampling guarantees $c_1 \in \mathcal{R}^{(1)}$.

Inductive step: Assume the theorem holds for $(k-1)$-level hierarchies. For a k-level hierarchy:

1. The first-level KVRM selects κ_i from $\{\kappa_1, ..., \kappa_m\}$ (valid by Lemma 1)
2. This selection routes to sub-registry $\mathcal{R}_i^{(k-1)}$
3. By induction, the $(k-1)$-level selection produces valid output in $\mathcal{R}_i^{(k-1)}$
4. The composition $\kappa_i \circ c_2 \circ ... \circ c_k$ is valid since each component is valid in its respective registry

The invalid output probability is $\prod_{i=1}^{k} P(\text{invalid at level } i) = \prod_{i=1}^{k} 0 = 0$. □

Complexity Reduction Formula:

- Single-stage: $C_{\text{single}} = N$ (must learn N-way classification)
- k-level hierarchical: $C_{\text{hier}} = \sum_{i=1}^{k} n_i$ where $\prod n_i = N$

For $N = 1000$: $C_{\text{single}} = 1000$ vs $C_{\text{3-level}} = 10 + 10 + 10 = 30$ (33x complexity reduction).

5.9 KVRM vs Hierarchical Softmax

Table 14: KVRM Hierarchical vs H-Softmax (3-level, 1000 classes)

Aspect	**H-Softmax**	**KVRM Hierarchical**
Training	End-to-end	Progressive + domain
Context enrichment	None	Previous selections
Accuracy (3-level)	1.5% ± 0.1%	**14.5% ± 0.8%**
Improvement	–	**9.9x** ($p < 0.0001$)
Registry versioning	No	Yes

Why KVRM outperforms: The **9.9x improvement** comes from context enrichment. Each level's selection is embedded and concatenated with the encoded input to inform the next level's decision. H-Softmax makes independent predictions at each level.

5.10 Probabilistic Rejection Bounds

Definition 9 (Verified Rejection): A KVRM network with verified rejection augments the registry with $\perp$ (abstain) and rejects when entropy exceeds threshold τ:

$$f_\theta^{VR}(x) = \begin{cases} f_\theta(x) & \text{if } H(z) \leq \tau \\ \perp & \text{if } H(z) > \tau \end{cases}$$

Table 15: Verified Rejection Tradeoffs

Threshold τ	**Rejection Rate**	**Accuracy**	**Min Confidence**
0.5	45.2%	99.1%	60.7%
1.0	18.7%	97.8%	36.8%
1.5	5.3%	96.2%	22.3%
∞	0%	96.0%	N/A

Key Finding: At $\tau = 0.5$, rejecting 45% of uncertain inputs raises accuracy from 96% to 99.1%.

6 Ablation Study

*Progressive training shows negative contribution on simple tasks; benefits emerge on complex tasks (1000+ classes).

Key Finding: Data constraint provides the largest improvement (+72.4%). Structural constraint guarantees 0% invalid regardless of data quality.

Table 16: Three Pillars Ablation (3 seeds)

Configuration	Accuracy	Invalid	Contribution
Structural Only	25.5% ± 15.7%	**0.0%**	Baseline
+ Data Constraint	97.9% ± 0.2%	**0.0%**	**+72.4%**
+ Progressive Training	89.8% ± 0.7%	**0.0%**	−8.1%*

7 When Does KVRM Fail?

7.1 Known Failure Modes

KVRM is not universally beneficial. Table 17 summarizes identified failure modes where KVRM underperforms relative to simpler approaches.

Small Registries (<50 entries): When the output space is small, well-designed heuristics often match KVRM performance with lower infrastructure cost. The overhead of training and deploying a neural network is not justified when a simple decision tree or rule-based system achieves comparable results. The recommendation is to use heuristics for registries with fewer than 50 entries.

Highly Structured Tasks: For tasks with highly predictable structure (e.g., selecting from a small set of mutually exclusive options), KVRM provides no learning advantage over deterministic algorithms. Examples include selecting among 3-4 well-characterized kernel configurations where performance characteristics are known *a priori*. In such cases, KVRM's complexity is not warranted.

Attention Kernel Selection: A notable empirical failure is attention kernel selection, where KVRM shows 62% higher regret than the heuristic baseline. Analysis indicates that attention kernel performance is highly input-dependent and exhibits non-stationary patterns that the KVRM model fails to capture. The heuristic (which uses sequence length as a proxy) is more robust to this variation. The recommendation is to use the heuristic for attention kernels while retaining KVRM for GEMM, LayerNorm, and softmax kernels where it shows consistent gains.

When to Use KVRM: KVRM is recommended for (1) medium to large registries (100+ entries), (2) tasks with complex input-dependent selection criteria, and (3) scenarios where validity guarantees are critical. KVRM should be avoided for small registries, highly structured tasks, and specific failure modes identified through empirical validation.

Table 17: KVRM Failure Modes

Mode	Symptom	Mitigation
Small registries (<50)	Heuristics match KVRM	Use heuristics
Highly structured tasks	No learning advantage	Skip neural selection
Attention kernel selection	−62% regret vs heuristic	Use heuristic for attention

7.2 Infrastructure Overhead

Measured overhead: 2.1x infrastructure overhead for continuous batching integration. This overhead comes from (1) selector network forward pass (<0.3ms), (2) kernel dispatch coordination, and (3) context switching between Python control plane and GPU compute.

Breakeven Analysis: At 2.1x overhead, KVRM needs >52% end-to-end speedup for net benefit. Current aggregate algorithmic speedup is 1.42x (before overhead), meaning end-to-end speedup is

approximately 1.42x / 2.1x ≈ 0.68x (a net slowdown). This candid assessment shows KVRM is not yet production-ready for latency-critical deployment without further optimization.

Detailed Optimization Roadmap:

1. **Phase 1 (Current - Proof of Concept):** Demonstrates algorithmic validity and regret reduction on synthetic benchmarks. Focus is on correctness, not performance. Infrastructure overhead is acceptable for research validation.

2. **Phase 2 (Kernel Fusion - Target: 1.3x overhead):** Implement CUDA kernel fusion to eliminate context switching. Instead of Python → selector → Python → kernel, implement fused kernels that embed selection logic directly in the launch schedule. Estimated development effort: 2-3 months with CUDA expertise.

3. **Phase 3 (GPU-Accelerated Control - Target: 1.1x overhead):** Move selector network inference to GPU using TensorRT or TorchScript. Eliminate CPU-GPU synchronization points. Implement asynchronous selection to overlap with compute. Estimated development effort: 3-4 months with systems expertise.

4. **Phase 4 (Production Deployment - Target: <1.05x overhead):** Integrate with existing serving systems (vLLM, TensorRT-LLM). Implement caching for recurrent contexts. Optimize for batch-1 serving. This phase requires production engineering and validation on real workloads.

Path to Production Viability: At Phase 3 (1.1x overhead), KVRM achieves net speedup of 1.42x / 1.1x ≈ 1.29x, making it viable for deployment. At Phase 4 (<1.05x overhead), net speedup exceeds 1.35x, providing meaningful production value. The roadmap is technically feasible but requires significant engineering effort beyond this research paper.

8 Threats to Validity

The following limitations are identified that may affect the generalizability of the findings:

8.1 Internal Validity

Synthetic benchmarks: The evaluation uses synthetic GPU kernel benchmarks rather than production workloads. Real-world workloads may exhibit different patterns that affect KVRM's regret reduction.

Oracle construction: The "oracle" baseline uses exhaustive search, which may not represent true optimal selection in all cases due to measurement noise.

Seed sensitivity: Results are reported across multiple seeds ($n = 3$ to $n = 10$), though some variance remains (e.g., KVRM accuracy: 96.0% ± 2.6%).

8.2 External Validity

Hardware scope: Validated on H200 NVL (using LLM workloads including Mistral-7B and Qwen3-8B for kernel characterization), RTX 4090, and Apple MPS. Results may not transfer to other GPU architectures (e.g., AMD, Intel) or accelerators (TPU, NPU).

Model scale: KVRM tile selection is model-agnostic at the kernel level (optimizes attention, GEMM, LayerNorm, softmax kernels independently of which LLM is running). Hardware validation used 7-8B parameter models; kernel performance characteristics may differ for significantly different architectures, but the tile selection mechanism itself applies regardless of model scale.

Registry size bounds: Hierarchical registries tested to 10K entries. Scaling behavior beyond 100K entries is untested.

8.3 Construct Validity

Regret definition: Regret is defined as $\frac{\text{selected}-\text{oracle}}{\text{oracle}}$. Alternative definitions (e.g., rank-based) may yield different conclusions.

"Valid" vs "correct": KVRM guarantees output $\in$ registry (structural validity), *not* optimal selection (semantic correctness). The 4.0% "valid-but-wrong" rate represents this gap.

8.4 Statistical Validity

Sample sizes: Downstream task evaluations use limited samples ($n = 50$-150). Statistical power for detecting small effects is limited.

Multiple comparisons: Evaluation is done across 4 kernel types without formal multiple comparison correction. Per-kernel p-values should be interpreted cautiously.

9 Novel Evaluation Metrics

This section introduces two novel metrics for evaluating constrained neural selection systems: the Verification Overhead Ratio (VOR) and the Compositional Safety Score (CSS). These metrics are designed to capture trade-offs specific to validity-constrained systems that traditional metrics (accuracy, latency) do not adequately address.

9.1 Verification Overhead Ratio (VOR)

Traditional neural network evaluation focuses on accuracy and latency, but does not account for the cost of ensuring output validity. In safety-critical systems, invalid outputs must be detected and corrected, which imposes verification overhead. VOR quantifies this trade-off:

$$\text{VOR} = \frac{\text{Registry Size} \times \text{Training Time}}{\text{Accuracy} \times (1 - \text{Invalid Rate})}$$

Lower VOR indicates better cost-benefit ratio: the system achieves high accuracy with low validity overhead. A VOR < 1.0 indicates that verification is "free" in cost-benefit terms, meaning the benefits outweigh the costs.

Key Finding: KVRM achieves VOR < 1.0 for registries up to approximately 50 options. The 0% invalid rate (architectural guarantee) eliminates the denominator penalty that plagues unconstrained systems. For larger registries (>100 options), VOR increases due to longer training times, though the 0% invalid rate provides a consistent advantage.

Comparison: Standard neural networks with post-hoc validation typically achieve VOR > 2.0 due to the combined cost of high invalid rates (5-20%) and additional verification infrastructure. Grammar-based constrained decoding achieves VOR ≈ 1.5 due to runtime validation overhead (1-5ms latency).

9.2 Compositional Safety Score (CSS)

Hierarchical composition introduces a new challenge: validity at each level does not guarantee validity of the composed system. CSS quantifies safety in hierarchical systems:

$$\text{CSS} = \prod_{i=1}^{k}(1 - \text{InvalidRate}_i) \times \frac{\text{Accuracy}_{\text{composed}}}{\text{Accuracy}_{\text{oracle}}}$$

CSS ranges from 0 (unsafe) to 1 (perfectly safe). The product term captures the compounding effect of invalid rates across hierarchy levels: even a 1% invalid rate at each level results in $1 - 0.99^k$ composite invalid rate.

Key Finding: KVRM's architectural validity guarantee means the safety factor is always 1.0, regardless of composition depth. The $\prod(1 - \text{InvalidRate}_i)$ term equals 1.0 because $\text{InvalidRate}_i = 0$ at all levels. This is a key advantage over behavioral approaches where validity degrades with hierarchy depth.

Comparison: For a 4-level hierarchy with 1% invalid rate at each level, behavioral approaches achieve CSS ≈ 0.96 (4% composite invalid rate). KVRM maintains CSS = 1.0 regardless of hierarchy depth.

10 Broader Impacts

Benefits: Verified outputs enable safer neural network deployment in system-critical applications. KVRM's architectural validity guarantee (0% invalid outputs) eliminates a class of failures that can cause crashes, corruption, or undefined behavior in production systems. This is particularly valuable for:

- **Safety-critical systems:** Medical devices, automotive systems, aerospace control where invalid outputs could endanger life
- **Financial systems:** Trading systems, risk analysis where invalid parameters could cause significant losses
- **Infrastructure:** Cloud orchestration, network configuration where invalid settings could cause outages

Democratization of safe AI: By providing a systematic methodology for validity-constrained neural selection, KVRM enables smaller organizations and research groups to deploy neural networks in safety-sensitive contexts without access to expensive verification infrastructure.

Risks: Could enable more autonomous systems with reduced human oversight. This is mitigated by (1) focusing on discrete selection from pre-validated registries (not open-ended generation), (2) the architectural guarantee only covers validity (not correctness), so human oversight is still required for optimal performance, and (3) publishing for defensive adoption to enable security researchers to understand and validate the approach.

Not ready for safety-critical deployment: Despite the 0% invalid output guarantee, KVRM is not yet recommended for unsupervised deployment in safety-critical systems. The 4.0% valid-but-wrong rate means the system will make suboptimal selections that could degrade performance. The 2.1x infrastructure overhead means net performance may not improve without further optimization (see Section 8.2). Production deployment requires: (1) extensive validation on domain-specific workloads, (2) optimization to reduce infrastructure overhead, (3) human-in-the-loop monitoring during initial deployment, and (4) fallback mechanisms for edge cases.

Data and reproducibility: All experiments use synthetic benchmarks; no personal data or proprietary information was used. Full source code, training scripts, and experimental data are available on GitHub with detailed reproduction instructions, enabling independent verification and extension of these results.

11 Reproducibility

Complete source code, training scripts, and experimental data are available at: `https://github.com/bprice-code/kvrm-gpu`. The results in this paper correspond to commit hash `44901644d7ef77b28ed70e6f4a5983ee` (January 2026).

Quick reproduction: Run `make reproduce-kvrm` to reproduce all core KVRM experiments (∼2 hours on H200, ∼4 hours on RTX 4090). This command trains all selector networks and reproduces the key results reported in this paper.

Experiment organization: The codebase is organized with separate directories for each experimental domain: `experiments/tile_selection/`, `experiments/instruction_decoding/`, `experiments/api_selec` and `experiments/hierarchical/`. Each directory contains configuration files, training scripts, and evaluation notebooks.

Data availability: All experimental results (CSV files with raw measurements) are included in the repository. No external datasets are required beyond the synthetic benchmarks generated by the evaluation scripts.

Dependencies: Python 3.11+, PyTorch 2.9+, CUDA 12.8 (for NVIDIA GPUs). The code is pure Python/PyTorch with no custom CUDA kernels, ensuring portability across platforms.

11.1 Artifact Checklist

Requirement	Status	Notes
Single command reproduces all	Yes	`make reproduce-all`
Deterministic seeds	Yes	Seeds [42, 43, 44]
Environment specification	Yes	Python 3.10+, PyTorch 2.0+
Expected runtime	Yes	2-4 hours

References

[1] Fawzi, A., et al. (2022). Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930).

[2] Chen, T., et al. (2018). TVM: An automated end-to-end optimizing compiler. *OSDI 2018*.

[3] Zheng, L., et al. (2020). Ansor: Generating high-performance tensor programs. *OSDI 2020*.

[4] Lange, R., et al. (2025). GPU Kernel Scientist: Multi-stage evolutionary optimization. *arXiv:2506.20807*.

[5] Wang, X., Awad, M. (2025). Omniwise: Predicting GPU Kernel Performance with Large Language Models. *arXiv:2506.20886*.

[6] Willard, B., Louf, R. (2024). Efficient Guided Generation for LLMs. *arXiv:2307.09702*.

[7] Dong, Y., et al. (2025). XGrammar: Flexible structured generation. *arXiv:2411.15100*.

[8] Zhang, S., et al. (2023). H2O: Heavy-Hitter Oracle. *NeurIPS 2023*.

[9] Morin, F., Bengio, Y. (2005). Hierarchical probabilistic neural network language model. *AISTATS 2005*.

[10] Schick, T., et al. (2023). Toolformer: Language models can teach themselves to use tools. *NeurIPS 2023*.